Práctica 2: Arco & Flecha 2.0

Curso 2026-2027. Tecnología de la Programación de Videojuegos 1. UCM

📅 Fecha de entrega: 16 de noviembre de 2026

El objetivo fundamental de esta práctica es incorporar el uso de la herencia y el polimorfismo en la programación de videojuegos mediante C++/SDL. Para ello, partiremos de la práctica anterior y desarrollaremos las modificaciones que se explican a continuación. En cuanto a funcionalidad, el juego presenta las siguientes novedades:

1. Se añade un nuevo elemento al juego, la mariposa (🦋), que vuela de forma aleatoria por la ventana. Al principio del nivel se crea un número determinado de mariposas y estas mueren al ser alcanzadas por las flechas, restando una puntuación fija al jugador.
2. El juego funciona ahora como una sucesión de niveles. Cuando se alcance la puntuación p indicada por el atributo `goal` p de la especificación del nivel n`.txt`, se pasará al nivel siguiente $n+1$`.txt`. El primer nivel es $n = 1$ y si el siguiente nivel no existe se acaba la partida con victoria del jugador. Una puntuación $p = 0$ significa que nunca se pasa al siguiente nivel.
3. Se ha de otorgar puntuación extra por explotar más de un globo con una misma flecha. A la puntuación habitual por explotar un globo se puede añadir, por ejemplo, una cantidad multiplicada por el número de globos explotados por la fecha hasta el momento.
4. Se podrá cargar y guardar partida en cualquier momento del juego con las combinaciones de teclas Ctrl + O y Ctrl + S, respectivamente.

⚠️ Este enunciado no es una guía de implementación paso a paso y se complementa con las explicaciones y aclaraciones dadas en clase. Se ha de atender a lo visto en el tema 4 (herencia y polimorfismo), 5 (arquitectura de un videojuego) y el resto del temario. Lo demás queda a la libre elección e interpretación.

Detalles de implementación

Diseño de clases

El diagrama de la figura 1 muestra las clases obligatorias de esta práctica, sus relaciones de herencia pública y algunos de sus atributos y métodos. El nombre de los atributos es libre y solo se muestra con fines ilustrativos. Además, deberás utilizar los atributos e implementar los métodos (y posiblemente las clases) adicionales que consideres necesarios para mejorar la claridad y reusabilidad del código.

Clase	Descripción	Atributos (📦) y métodos (☸)
`GameObject`	Objeto cualquiera del juego que se sabe pintar y actualizar (clase abstracta).	📦 Puntero a `Game` ☸ `void render() const = 0` ☸ `void update() = 0`
`SceneObject`	Objeto presente en la escena que puede chocar con la flecha (clase posiblemente abstracta). La lista de objetos de `Game` será de este tipo.	📦 Posición y dimensiones ☸ `bool hit(const Rectangle&)`
`MobileObject`	Objeto de la escena (`SceneObject`) dotado de velocidad y que se mueve de acuerdo a ella.	📦 Velocidad ☸ Sobrescritura de `update`
 `Butterfly`	Mariposa que se mueve por la ventana y resta puntos al ser acertada por la flecha. Véase § *Generación y movimiento de las mariposas*.	Los que sean necesarios.

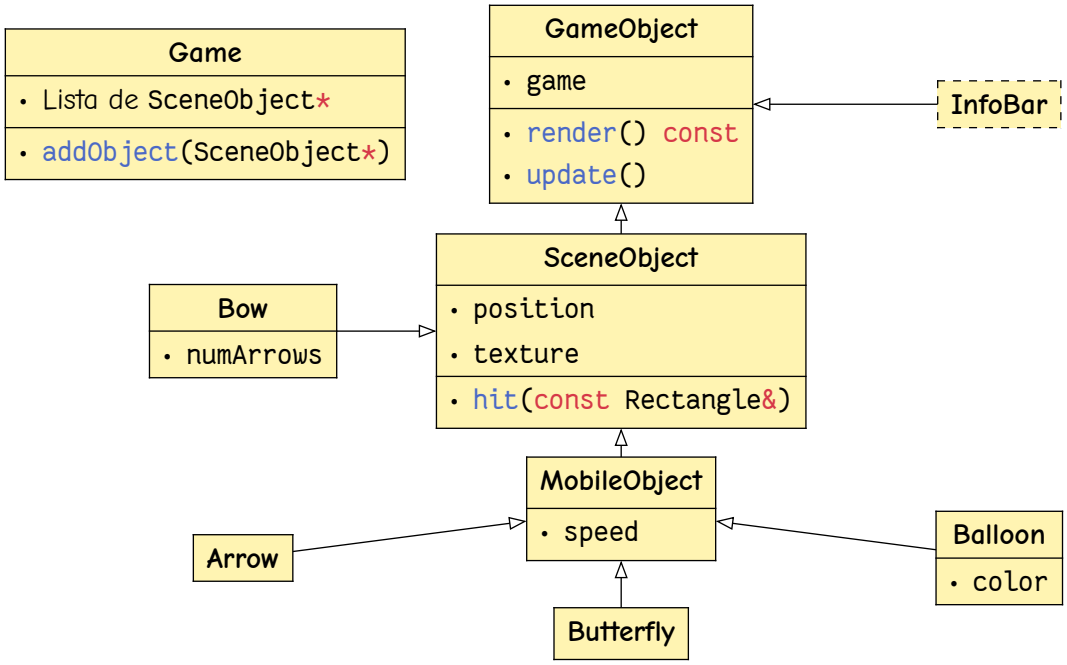


Figura 1: Diagrama de clases de la práctica 2.

Las clases `Arrow` y `Balloon` serán ahora subclases de `MobileObject`, mientras que `Bow` lo será de `SceneObject`. Si existe, la clase opcional `InfoBar` de la práctica 1 será subclase de `GameObject`, ya que no puede colisionar con la flecha. `Game` se adaptará para manipular los objetos explotando el polimorfismo como se explica en § *Lista polimórfica de objetos del juego*.

Lista polimórfica de objetos del juego

En lugar de almacenar múltiples vectores para cada tipo de objetos del juego, la clase `Game` mantendrá una lista polimórfica de punteros a `SceneObject`. Se ha de utilizar el tipo plantilla `list` de la STL. Los métodos `render`, `update` y `checkCollision` solo tendrán que recorrer esa única lista para aplicar las operaciones correspondientes. Los objetos de tipo `GameObject` que no sean `SceneObject` se pueden mantener por separado por simplicidad. Además, sin perjuicio de su pertenencia a la lista, aquellos elementos a los que se necesite acceder directamente (tal vez `Bow`) pueden tener su atributo propio.

La eliminación de los objetos de la lista se hará como en la práctica anterior, consultando el valor booleano devuelto por el método `update` del `GameObject`. El recorrido de actualización sobre la lista polimórfica deberá usar un iterador para poder llamar al método `erase` de la lista y eliminar el objeto afectado.

Generación y movimiento de las mariposas

A diferencia de los globos, las mariposas no se generan durante el juego, sino todas juntas al arrancar el nivel. Cada mariposa se generará en una posición y con una velocidad inicial aleatorias.

Cuando una mariposa trate de rebasar los límites de la ventana cambiará su dirección para alejarse del borde. Además, para evitar que la mariposa tenga un movimiento totalmente predecible, su dirección podrá cambiar aleatoriamente cada cierto tiempo. Cuando la mariposa es acertada por la flecha, su cadáver caerá a velocidad constante hacia el extremo inferior de la ventana y se destruirá.

Formato de configuración de niveles

El formato de configuración de niveles introducido en la práctica anterior se vuelve más interesante con los cambios de nivel. Además de los parámetros `arrows` y `baldelay`, en esta práctica se han de admitir:

- `goal` n indica que la puntuación requerida para pasar de nivel es n. Si $n = 0$, el nivel solo acaba cuando el jugador gasta todas sus flechas.
- `bgcolor` r g b establece el color de fondo del nivel al valor RGB (r, g, b) con $0 \leq r, g, b \leq 255$.

- `balspeed` v fija a $v > 0$ la velocidad base de ascenso de los globos, es decir, aquella sobre la que se aplicaba una variación aleatoria en la práctica anterior.
- `butterflies` n fija el número de mariposas iniciales en el nivel a n.

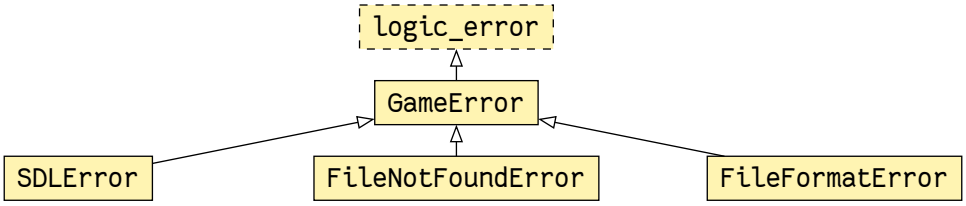
Guardado y carga de partidas

El guardado y la carga de partidas en esta práctica utilizará un archivo fijo `saved.txt`, que se escribirá al pulsar Ctrl + S y se leerá al pulsar Ctrl + O. El formato de este archivo es libre, aunque deberá incluir toda la información necesaria para recuperar el estado que se guardó (salvo los parámetros propios del nivel). Una posibilidad es utilizar un encabezado para los datos globales del juego (nivel, puntuación, etc.) y una línea adicional por cada objeto, con un símbolo o palabra que identifique su tipo seguido de sus parámetros. Por ejemplo, `B` x y w h u v c e puede ser la representación de un globo en la posición (x, y), de dimensiones (w, h), con velocidad (u, v), color c y estado e (explotado, no explotado, etc.).

La lectura y escritura de los objetos del juego se ha distribuir entre las clases correspondientes utilizando constructores o métodos virtuales, que reciban parámetros de tipo `std::istream&` o `std::ostream&`. Cada clase de la jerarquía se ha de ocupar de cargar o escribir la información que le es propia (por ejemplo, `SceneObject` deberá encargarse de sus propios atributos). Además, se deberá comprobar la validez de la entrada al menos en algún caso y lanzar una excepción si es preciso.

Jerarquía de excepciones

Los errores del juego se han de manejar mediante las siguientes clases de excepciones. Las excepciones irrecuperables se deben capturar en el `main` y la función `SDL_ShowSimpleMessageBox` puede utilizarse para mostrar el mensaje de error al jugador.



GameError: hereda de `std::logic_error` y sirve como superclase de todas las demás excepciones que definiremos. No proporciona nueva funcionalidad y simplemente reutiliza el constructor y método `what` de `logic_error`. Sin embargo, su tipo permite filtrar las excepciones propias del juego en bloques `catch`.

SDLError: hereda de `GameError` y se utiliza para todos los errores relacionados con la inicialización y uso de SDL. Utiliza la función `SDL_GetError` para obtener un mensaje específico sobre el error de SDL que se almacenará en la excepción.

FileNotFoundError: hereda de `GameError` y se utiliza para los errores provocados al no encontrarse un fichero que el programa trata de abrir. El mensaje del error debe incluir el nombre del fichero en cuestión.

FileFormatError: hereda de `GameError` y se utiliza para los errores provocados en la lectura de los archivos de datos del juego (niveles o partidas guardadas). La excepción debe almacenar y mostrar el nombre de archivo y el número de línea del error junto con el mensaje.

Pautas generales obligatorias

A continuación se indican algunas pautas generales que vuestro código debe seguir:

- Se debe hacer un buen uso de la herencia y del polimorfismo de manera que el código sea claro, se eviten las repeticiones de código, distinciones de casos innecesarias, no se abuse de castings ni de consultas de tipos en ejecución.

- Asegúrate de que el programa no deje basura revisando la salida de error o la consola de Visual Studio al finalizar el programa. Cuando se compila en modo *Debug*, allí aparecerá un volcado de todos las fugas de memoria detectadas.
- Asegúrate de que el programa hace un correcto uso de la memoria (se inicializan todas las variables y atributos antes de leerlos, no se libera dos veces un mismo bloque de memoria, etc.). El modo de compilación *Sanitize* activa un comprobador de memoria que puede detectar estos errores.
- Todos los atributos deben ser privados excepto quizás algunas constantes del juego definidas como atributos estáticos.
- Define las constantes que sean necesarias, mejor con `constexpr` que con `const`. En general, no deben aparecer literales que pudiesen corresponder con configuraciones del programa en el código.
- No debe haber métodos que superen las 30-40 líneas de código.
- Escribe comentarios claros en el código y explica especialmente los métodos o clases nuevos que hayas añadido. Sé cuidadoso también con los nombres que eliges para variables, parámetros, atributos y métodos. Es importante que denoten realmente lo que son o hacen. Usa preferiblemente nombres en inglés.

Funcionalidades opcionales

Suman como máximo un punto extra a la nota de la práctica, sin pasarse de diez. Cada funcionalidad cuenta según su dificultad y no es necesario completarlas todas para obtener el punto completo.

1. En lugar de guardar y cargar partida sobre un archivo fijo `saved.txt`, utiliza las funciones de la SDL `SDL_ShowOpenFileDialog` y `SDL_ShowSaveFileDialog` para que el usuario pueda seleccionar el archivo. Estas funciones son asíncronas, por lo que será necesario pausar la actualización del juego mientras se completa esta operación. Se explicarán más detalles en clase.
2. Si no quedan mariposas en la escena, se acaba la partida con derrota del jugador.
3. Utiliza el ratón como forma alternativa de controlar el movimiento del arco, cargar y lanzar las flechas. Los métodos para capturar la información del ratón se explicaron en las diapositivas del tema 3.
4. Haz que el juego reproduzca sonidos al lanzar una flecha, explotar de un globo, morir una mariposa, etc. Utiliza para ello el soporte para audio de SDL3, sobre el que puedes preguntar al profesor.
5. Cualquier funcionalidad opcional de la práctica 1 que no se hubiera contabilizado en la práctica anterior y no sea obligatoria en la práctica actual.

Entrega y defensa

En la tarea *Entrega de la práctica 2* del campus virtual y dentro de la fecha límite (ver junto al título), cualquiera de los miembros del grupo debe subir el código de la práctica siguiendo las instrucciones que allí aparezcan. La raíz de la solución debe incluir un archivo `info.txt` con los nombres de los componentes del grupo y unas líneas explicando las funcionalidades opcionales incluidas y/o las cosas que no estén funcionando correctamente.

Además, para que la práctica se considere entregada, se deberá acudir a la defensa en la sesión de laboratorio del martes 17 de noviembre. Allí se pedirá realizar una modificación o extensión de la práctica en los ordenadores del laboratorio durante un tiempo establecido y sin acceso a internet. Si no se consiguen los objetivos mínimos de esa modificación, la práctica se considerará suspensa.